

好的，这是一个非常棒的架构！您提供的文件完美地展示了一套专业、数据驱动、高度解耦的 UI 系统。它以 AttributeMenuWidgetController 为核心，巧妙地运用了 MVC（Model-View-Controller）设计模式、单例（Singleton）、数据资产（Data Asset）和 GameplayTags 来管理和显示属性信息。

下面，我们将根据 MVC 模式，详细描述这个系统的完整工作流程。

MVC 模式角色划分

首先，我们将您提供的文件和相关概念划分到 MVC 的三个层次中：

- **Model (模型 - 数据层)**：负责存储和管理所有数据。
 - **UAuraAttributeSet**: 存储**动态数据**，即角色当前真实的属性数值（例如力量是 15，韧性是 12）。
 - **UAttributeInfo (Data Asset)**: 存储**静态/描述性数据**，即属性的元信息（例如，力量这个属性的名字是"Strength"，它的描述是"Increases physical damage."）。
 - **FAuraGameplayTags (Singleton)**: 充当数据的“**钥匙**”。它提供了一种全局唯一的、无歧义的方式来引用各个属性，将动态数据和静态数据关联起来。
 - **View (视图 - 表现层)**：负责将数据呈现给用户。
 - **AttributeMenu Widget (蓝图)**: 这是用户直接看到的 UI 界面。它本身不包含任何逻辑，只负责根据指令显示文本和数值。
 - **Controller (控制器 - 逻辑层)**：作为模型和视图之间的桥梁。
 - **UAttributeMenuWidgetController**: **核心控制器**。它从模型层（AttributeSet 和 AttributeInfo）获取数据，处理并打包成视图层（Widget）可以直接使用的格式，然后通过委托（Delegate）将数据传递给视图。
 - **AAuraHUD**: **高级控制器/工厂**。它的职责是创建和初始化 AttributeMenuWidgetController 以及 AttributeMenu Widget，并将两者关联起来。
-

详细工作流程

第一步：基础构建 - 单例模式的 GameplayTags

系统的基础是 FAuraGameplayTags，它被设计成一个单例，以确保整个项目都使用同一套标签。

1. **定义单例:** 在 AuraGameplayTags.h 中，通过 static const FAuraGameplayTags& Get() 提供了一个全局访问点，并用一个静态成员 GameplayTags 来存储唯一的实例。
2. **初始化标签:**
 - 为了确保这些 C++ 中定义的“原生标签”在引擎启动时就被注册，系统重写了 UAssetManager 的 StartInitialLoading 函数。
 - 在 UAuraAssetManager::StartInitialLoading() 中，调用了 FAuraGameplayTags::InitializeNativeGameplayTags()。
 - InitializeNativeGameplayTags 函数使用 UGameplayTagsManager::Get().AddNativeGameplayTag() 来逐一注册所有属性标签（如 Attributes.Primary.Strength），并将返回的 FGameplayTag 存储在单例的成员变量中（如 GameplayTags.Attributes_Primary_Strength）。

结果: 现在，项目的任何地方都可以通过 FAuraGameplayTags::Get().Attributes_Primary_Strength 来获取一个准确无误、无需手打字符串的 FGameplayTag。

第二步：数据驱动 - 使用数据资产存储属性信息

为了避免将 UI 文本硬编码在代码中，系统使用了 UDataAsset。

1. **定义数据结构:** AttributeInfo.h 中定义了 FAuraAttributeInfo 结构体，它将一个 FGameplayTag 与 UI 所需的文本（名字、描述）关联起来。
2. **创建数据资产:** UAttributeInfo 类继承自 UDataAsset，它包含一个 FAuraAttributeInfo 的数组。开发者可以在 UE 编辑器中创建一个 AttributeInfo 的数据资产实例，并在其中像填写表格一样，为每个属性标签（例如从 FAuraGameplayTags 中选择 Attributes.Primary.Strength）配置对应的名字和描述。
3. **提供查询接口:** UAttributeInfo 提供了一个 FindAttributeInfoForTag 函数，允许控制器通过传入一个 GameplayTag 来方便地查找并返回对应的完整信息结构体。

第三步：MVC 协同工作 - 从数据显示到 UI

这是整个系统的核心运转流程，以 AAuraHUD 的初始化开始。

1. HUD 进行初始化 (在 AAuraHUD::InitOverlay 中):

- AAuraHUD 负责创建 AttributeMenu Widget 和 AttributeMenuWidgetController。
- 它首先创建 Widget。
- 然后, 它调用 GetAttributeMenuWidgetController()来创建或获取 Controller 实例。
- 在创建 Controller 时, 它将关键的游戏世界对象 (玩家控制器、玩家状态、ASC、AttributeSet) 打包成一个 FWidgetControllerParams 结构体并传递给 Controller。
- 最后, 它将创建好的 Controller 设置给 Widget, 并将两者连接起来 (OverlayWidget->SetWidgetController(WidgetController)在您的代码中展示了类似逻辑)。

2. Controller 广播初始值 (在 UAttributeMenuWidgetController::BroadcastInitialValues 中):

- 一旦 Controller 被初始化, 它就需要将所有属性的当前状态显示在 UI 上。
- **获取动态数据:** 它首先获取 UAuraAttributeSet 的引用, 这是其实时数值的来源。
- **获取“钥匙”:** 它通过 FAuraGameplayTags::Get().Attributes_Primary_Strength 获取到“力量”这个属性的唯一标签。
- **获取静态数据:** 它使用这个标签作为“钥匙”, 调用 AttributeInfo->FindAttributeInfoForTag(), 从 UAttributeInfo 数据资产中查找并获取包含“力量”这个名字和描述的 FAuraAttributeInfo 结构体。
- **合并数据:** 它将从 AttributeSet 中获取到的**实时数值** (AS->GetStrength()) 填入刚刚获取的 FAuraAttributeInfo 结构体的 AttributeValue 字段中。现在, 这个结构体包含了 UI 显示该属性所需的所有信息 (标签、名字、描述、当前值)。
- **广播给视图:** 最后, Controller 调用 AttributeInfoDelegate.Broadcast(Info), 将这个包含了所有信息的完整结构体广播出去。

3. 视图进行显示 (在 AttributeMenu Widget 蓝图中):

- AttributeMenu Widget 在其蓝图的 EventGraph 中，会获取其 WidgetController 的引用。
- 它会找到 AttributeInfoDelegate 这个委托，并对其进行**绑定 (Bind)**，通常是绑定到一个自定义事件上。
- 当 BroadcastInitialValues 被调用，AttributeInfoDelegate 被广播时，Widget 中绑定的自定义事件就会被触发。
- 该事件接收到传递过来的 FAuraAttributeInfo 结构体，然后它会分解这个结构体，将 AttributeName 设置到对应的文本框，将 AttributeValue 格式化后设置到另一个文本框，从而在屏幕上显示出“力量: 15”。

这个流程会对所有需要显示的属性重复执行，从而用初始值填充整个属性菜单。后续当属性发生变化时，BindCallbacksToDependencies 函数中的逻辑（您尚未实现）将会负责监听这些变化，并再次广播委托来更新 UI。